A Phase Vocoder Approach To Pitch Shifting

Sam Smith

April 15, 2026

1 Introduction

The VST (Visual Studio Technology) and AU (Audio Units) plugin ecosystem greatly contributes to the music industry today. With the rise of digital audio workstations (DAWs) in the early 2000s, technology has become an integral part of music production and composition [10]. In particular, the electric guitar has long been associated with technological advancements in the industry. Engineers first harnessed the power of vacuum tubes to create the earliest electric guitar amplifiers, and later transitioned to transistor-based designs.

Today, traditional amps and effect pedals have been overtaken by virtual amplifiers and software-based effects plugins. Once-dominant traditional hardware companies are now competing with highly technical and lucrative software companies that specialize in music production software [10]. The purpose of this project is to design and implement a high-demand audio plugin for modern music production software.

The plugin in question is a pitch shifter, a widely used effect that digitally modifies an input signal in real time. Its core functionality is to increase or decrease the input signal's pitch by algorithmically altering every frequency component in a signal [9]. More importantly, it allows for a guitar player to quickly change the tuning of their guitar. This solves the common problem of a guitar player wanting to play a particular song in a certin tuning but their guitar does not support said tuning. Thus, the pitch shifter solves this problem by digitally modifying the guitar signal to the desired tuning before it reaches the amplifier. This is especially useful for guitarists who want to detune their guitar (tune the guitar to a lower pitch). Before detuning a guitar, one must consider a number of physical factors, such as the strings' gauge (thickness), the width of the nut, and the length and tension of the neck. If even one of these factors does not support a major detuning, the guitar cannot be safely or effectively retuned. A pitch shifter removes these physical limitations by solving the tuning problem through a purely software-based approach.

1.1 Pitch Shifting

Pitch shifting is a desirable signal processing effect for a variety of practical reasons. Historically, pitch manipulation emerged from research in vocal processing (giving rise to the term 'vocoder' a concatenation of the words 'vocal' and 'coder') [2]. Today, pitch-shifting algorithms are widely used in music production, film, voice acting, and media. Specifically in music

production, digital vocal correction tools such as autotuning rely on pitch modification techniques to adjust a singer's intonation while preserving timing and vocal characteristics.

Pitch shifting is not limited to vocal signals, however, it is also commonly applied to instrumental audio. For example, electric guitarists use pitch-shifting pedals and digital audio plugins to alter tuning without physically adjusting string tension. This capability allows musicians to explore alternate tunings without modifying their instrument hardware or spending vast amounts of time retuning extended ranged guitars. Such applications highlight the broad practical demand for high-quality, real-time pitch-shifting algorithms.

1.2 The Phase Vocoder Approach

The phase vocoder is a signal processing technique used for both time-scale and pitch modifications. This algorithm was chosen to be the core processing algorithm for this project due to the fact that the family of vocoder algorithms is known for their reliability, ability to handle polyphonic signals, and their widespread adoption many modern signal processing applications. Furthermore, the phase vocoder is a real-time algorithm, meaning it has the capability to execute, in its entirety, in an unperceivable amount of time. Hence, this algorithm, along with the JUCE audio plugin framework, will provide a basis for creating a real-time pitch-shifting audio plugin.

1.3 Outline of Remaining Chapters

This paper begins with a comprehensive overview of the mathematics needed to understand the phase vocoder algorithm. It will begin with an explanation of the Discrete Fourier Transform, the backbone of signal processing, then we will examine windowing, why we need it and what problems it solves. Finally, we will examine the Short-Time Fourier Transform (STFT) which is the core analysis algorithm used in the phase vocoder.

The next section will provide a deep intuition for the phase vocoder algorithm where the goal is to connect the mathematical concepts highlighted in the background section to the processing chain that comprises the phase vocoder algorithm. The section will be divided based on the three fundamental steps that make up the phase vocoder algorithm: analysis, modification and synthesis.

Next, we will translate the mathematical concepts built up by the previous sections into a coherent program architecture. The audio plugin will be constructed in the JUCE framework, an open-source c++ framework designed to build audio plugins and and applications,

Finally, the following will evaluate output quality using metrics such as tonality and spectral flatness (noise) will be evaluated against to test how good the phase vocoder is at modifying a given input signal.

The conclusion follows the results chapter and will primarly focus on results, future work and optimizations that can be done on this project. Additionally, this section will provide insight into any errors or unoptimized processes in this project and offer any additional improvements or next steps.

2 Background

2.1 Discrete Fourier Transform

The Discrete Fourier Transform (DFT) is a fundamental tool used in digital signal processing that enables sophisticated analysis and synthesis of discrete-time signals [7]. The DFT transforms a signal from the time domain into its frequency-domain representation, allowing the signal to be expressed as a sum of sinusoids [7]. The DFT accepts a finite sequence of N discrete samples and produces N complex-valued outputs, each representing the signal's magnitude and phase at one of N evenly-spaced frequency bins. The output of the DFT can be better understood as a sequence of N complex numbers representing amplitude and phase for each analysis frequency [8]. Mathematically, the DFT is defined as:

$$X[m] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi n \frac{m}{N}} \tag{1}$$

where N is the number of samples from the input signal $x[n]$, n denotes the time-domain index, m represents the frequency bin index, n is the index of the input sequence in the time-domain and $j = \sqrt{-1}$. Each coefficient in $X[m]$ is complex-valued and can be expressed in terms of magnitude and phase, which describe the contribution of a corresponding frequency component to the overall signal [8]. Another important property of the DFT is that it is invertible; meaning a signal can be transformed from the frequency domain to the time domain. It can be described mathematically as:

$$x[n] = \frac{1}{N} \sum_{n=0}^{N-1} X[m]\, e^{j2\pi n \frac{m}{N}} \tag{2}$$

Since the forward direction of the DTF breaks a signal into its fundamental sinusoids, the inverse DFT perfoms the opposite. It computes each component of a signal $x[n]$ as the average of the nth samples of the DFT's fundamental sinusoids [8]. The inverse DFT is a crucial component of the phase vocoder as it allows for a signal to be transformed back to the time domain after being modified in the frequency domain.In practice, the DFT is much too slow for modern applications. Therefore, applications today use the Fast Fourier Transform (FFT), an optimized algorithm for computing the DFT. A common method is the Cooley and Tukey algorithm, which requires that the input signal length be a power of two [8].

2.2 Windowing

Windowing is a function used to isolate parts of a discrete time signal so that analysis on the signal can be performed on contiguous blocks [1]. More specifically, the input signal is multiplied by a windowing function whose domain is nonzero for a particular range. This results in a small localized snapshot of the input signal where every value outside of the window's domain is reduced to zero [12]. Figure 1 below demonstrates the functionality of a window function.

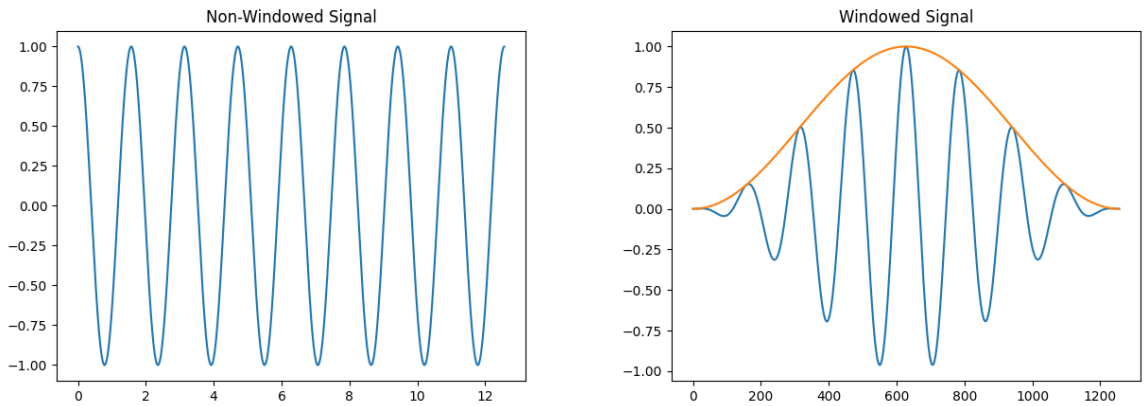

Figure 1: After applying a window function (highlighted by the orange cosine curve), the signal is tapered at the edges, resulting in a temporally localized signal.

As depicted by Figure 1, after applying the window function to the example signal, the edges of the signal tend toward zero. This byproduct of the window function is crucial for the phase vocoder. Without windowing, the signal may be discontinuous at its edges, meaning the beginning and end do not line up. These discontinuities cause spectral leakage in the frequency domain. This leakage manifests itself as sidelobes and causes the calculated frequency of the signal to be less accurate [11].

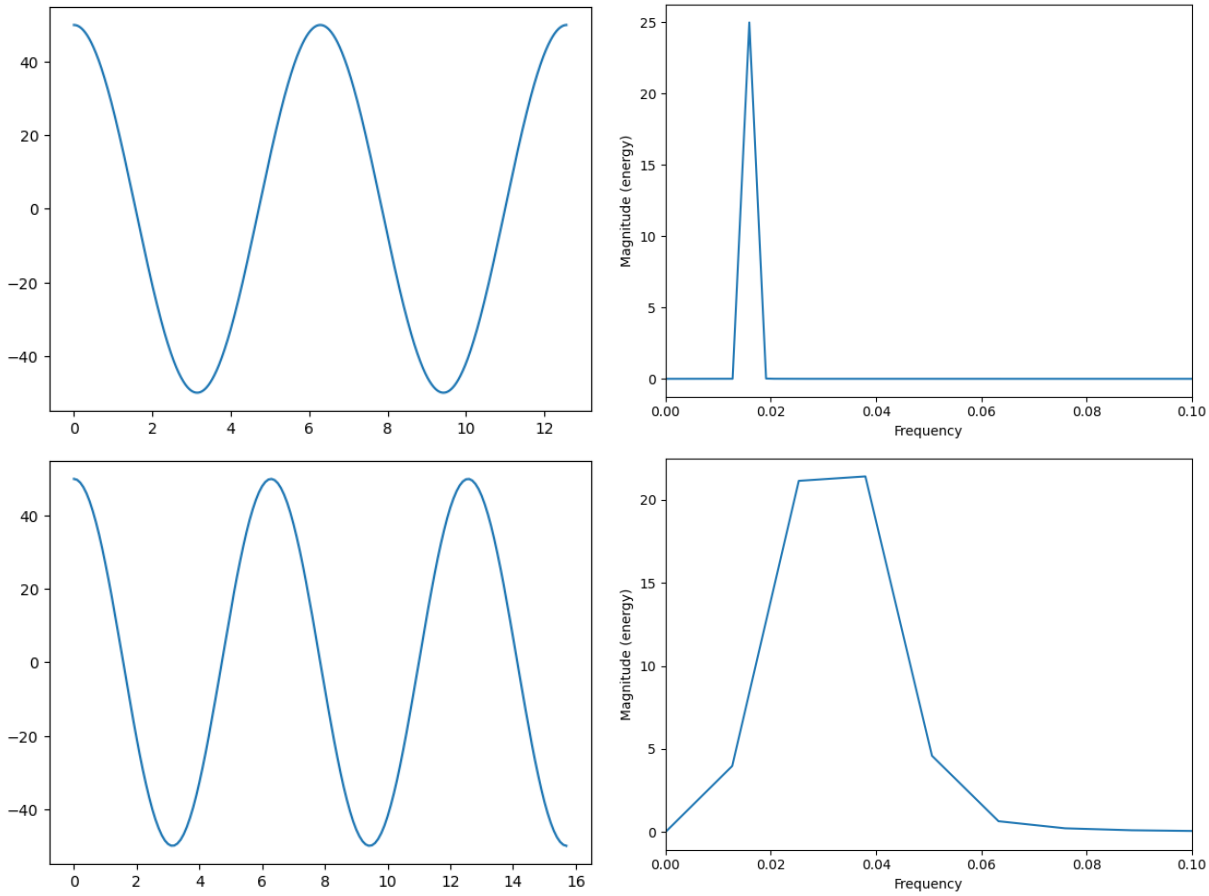

Figure 2: Illustration of spectral leakage. The top row shows a continuous signal segment that begins and ends at the same value, resulting in an accurate frequency representation. The bottom row shows a discontinuous signal segment, where the endpoints do not align, leading to spectral leakage and a spread in the frequency-domain representation.

In order to mitigate the negative consequences a discontinuous signal produces, a window function is used to force continuity at the signal's edges. In Figure 1, the Hanning window function was used to shape the signal, however, there are multiple windowing functions such as the Rectangle window function, Triangle window function and Hamming window function [8]. For this project, the Hanning window will be employed, as it produces a smoother and more continuous signal during the resynthesis step [3].

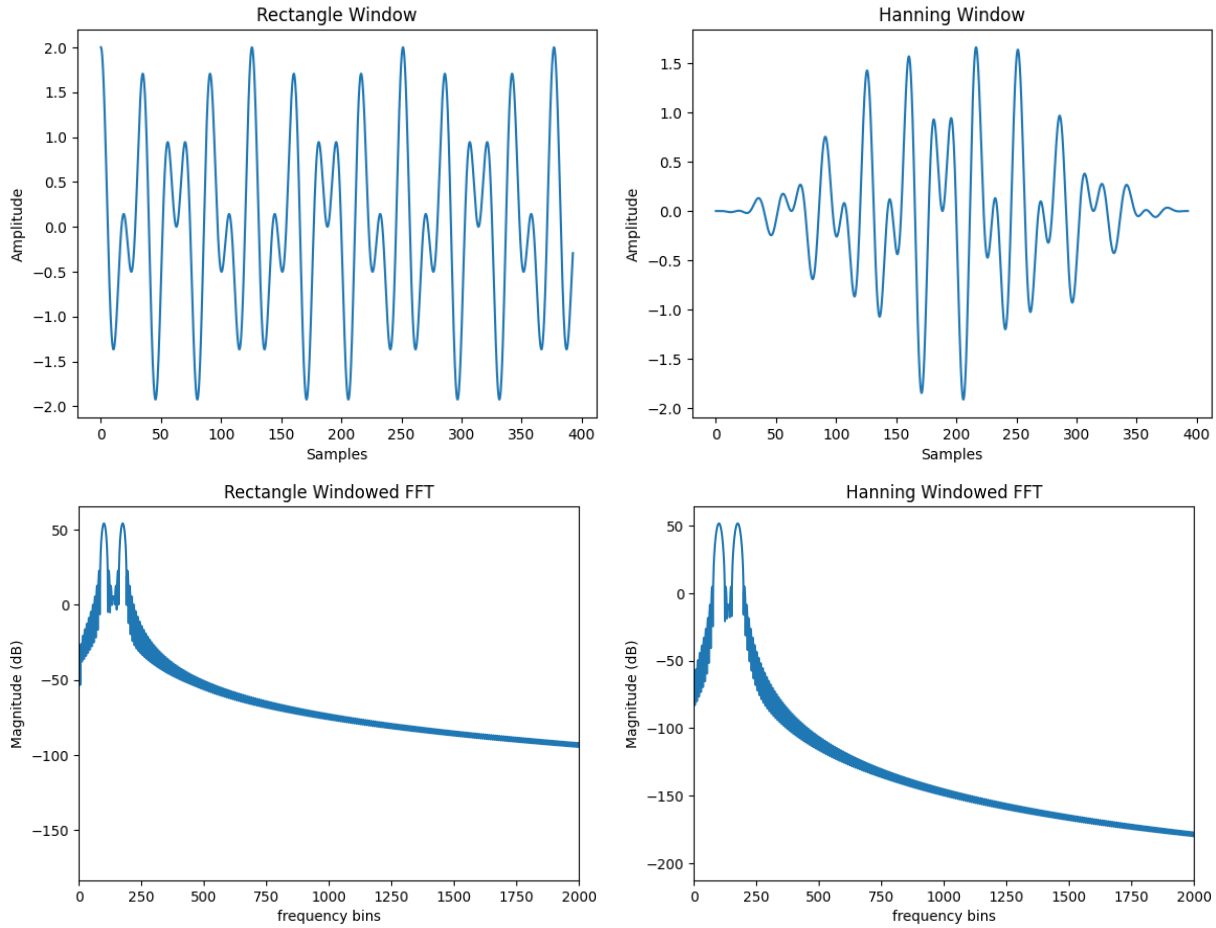

Figure 3: Illustration of the windowing effect. The top row shows the rectangle windowed and Hanning windowed signals. The bottom row shows their corresponding frequency plots. Applying the Hanning window reduces spectral leakage, making the main frequency component more accurately represented.

Notice that the Hanning frequency plot tapers off much more rapidly than the rectangle frequency plot. The regions of a frequency spectrum outside of the main peeks are referred to as side lobes. If the side lobes are too prominant in the windowed function, subsequent frequency estimations will be less accruate.

2.3 Short-Time Fourier Transform

The Short-Time Fourier Transform (STFT) is a time-frequency representation used to compute the DFT of contiguous windowed segments of a signal. In practice, however, the STFT uses the FFT as this is more computationally efficient. This is accomplished by multiplying the signal by a windowing function so that only a small portion of the signal is nonzero [2]. Mathematically, it is similar to equation (1) but with a few modifications: the addition of a windowing function, the domain of the input signal, and how the output is interpreted.

Below is the STFT:

$$X[m,k] = \sum_{n=0}^{N-1} x[n+kH]w[n]e^{-2\pi jmn/N} \tag{3}$$

Where $X[m,k]$ is the 2-dimensional output array indexed by both frequency bin and time frame and $x[n+kH]$ is the section of the input signal determined by frame index and hop size [12]. The outcome of the STFT is a representation that transforms a signal from the time domain to the frequency domain. In this form, the signal's frequency components can be systematically modified and recombined to reconstruct a new signal.

Since the STFT only computes the DFT of a particular section of a given signal, it must be shifted in order to process another section of the signal. The amount in which the STFT is shifted is known as the *Hop Size*, denoted as H. The hop size is determined by how much the successive STFTs should overlap, more on this in Section 4.3. Typical hop sizes include 75%, 50%, 33.3%, and 20% overlap [3]. Lastly, just as the DFT has an inverse, so to does the STFT. However, instead of reconstructing the entire signal, the inverse STFT only reconstructs a particular chunk of a signal. This is especially useful in reconstructing a signal after modifying it in the frequency domain.

3 Related Works

3.1 Classical Phase Vocoder Approaches

Pitch-shifting has been approached by a number of time–frequency methods. Early methods, such as Dolson's (1986), employed the use of the Short-Time Fourier Transform in conjunction with the phase vocoder algorithm. In particular, Dolson describes a pitch-shifting method that relies on time-scale manipulation to modify the pitch of a given signal [2]. For example, by stretching a signal, making it longer in the time domain, and then rescaling it back to its original length, the overall frequency content of the signal is effectively decreased, which corresponds to lowering the signal's pitch. Dolson acknowledges, however, that this approach is imperfect: specific constraints must be enforced when performing a pitch shift, as failing to do so may introduce unwanted artifacts into the signal or result in a reconstructed signal that has lost information [2]. While effective for monophonic (single note) pitch scaling, this classical time-scale modification framework treats spectral bins independently, which can lead to phase incoherence and unwanted artifacts. These limitations motivated subsequent refinements to the phase vocoder algorithm.

3.2 Phase Vocoder Optimizations

Unlike the original process chain described by Dolson (1986), which applies a uniform scaling factor to all spectral components, later work by Laroche and Dolson (1999) introduces refinements that improve frequency coherence [5]. In particular, they highlight a new peak-detection paradigm, where peaks refer to how much a particular frequency is in the input signal. Once the peaks are detected, they can be shifted around by a pitch-factor and unlike

the version described by Dolson (1986), the pitch-factor is not restricted to an integer amount [5].

In addition to optimizing frequency resolution, modern implementations of the phase vocoder shift away from the standard two-step timescale and resample approach and adopt an analysis and resynthesis paradigm [3]. The new approach leverages phase information obtained from the STFT step to more accurately reconstruct the modified signal [3]. More specifically, a phase adjustment value can be derived from the frequency bins across two frames and when added to the original frequency approximation the accuracy of the frequency estimation is increased [6].

3.3 Alternative Pitch-Shifting Methods

The prior sections highlight the phase vocoder approach to pitch-shifting, a frequency-domain algorithm that converts a signal from the time domain to the frequency domain to compute the pitch shift. An alternative approach uses time-scale modification (TSM) algorithms, which operate entirely in the time domain [9]. A common algorithm belonging to the TSM family is the Overlap-Add (OLA) algorithm [12].

The idea behind these algorithms is that pitch shifting and time scaling are essentially the same; meaning if you have one, either a pitch shifter or time scaler, then you effectively have the other. OLA makes use of this paradigm. The algorithm first partitions the input signal into frames, much like the STFT. Then, if the goal is to pitch down the signal, when resynthesizing the signal it is constructed in such a way that extra space is added between the reconstructed frames, increasing the length of the signal. On the contrary, if the goal is to make the signal's pitch higher, then the algorithm reconstructs the signal in such a way where the space between contiguous frames is reduced, effectively shortening the signal's length [12].

Since TSM algorithms operate in the time domain and thus change the input signal's duration, either lengthening or shortening it, these algorithms must incorporate a resample step in order to convert the modified signal back to the original signal's duration [9]. Thus, these alternative algorithms are viable options; however, purely TSM-based algorithms are often outperformed by frequency-based algorithms [12].

4 Phase Vocoder

The phase vocoder is a name given to a group of algorithms designed to process an input signal in the frequency domain [11]. This paper specifically focuses on the application of pitch shifting, the process of either increasing or decreasing an audio signal's pitch. In particular, this work utilizes a phase vocoder-based pitch shifting algorithm, as it is a computationally efficient method for analyzing and modifying an input audio signal, and is the backbone of many real-time pitch shifting audio plugins used today [12].

The following subsections will explain the inner workings of the phase vocoder algorithm and its application to pitch shifting. The first subsection will explain the analysis step and how the STFT breaks an audio signal into frames. The second subsection will cover the modification step and explain how pitch shifting is computed in the frequency domain.

Finally, the last subsection will explain how the modified signal is synthesized back into an output signal.

4.1 Analysis

The first step of the phase vocoder algorithm is the analysis step. The primary goal of this step is to break an input audio signal into a succession of overlapping frames so that each frame is processed individually. The analysis step segments the signal into frames because it is designed to operate in a real-time environment, such as an audio plugin. In this setting, the phase vocoder does not have access to the entire signal at once, as it is being continuously fed into the phase vocoder from a source. Thus, the phase vocoder is not concerned with the global scope of the signal, it only needs to process the signal locally.

In practice, the STFT is used to partition the audio signal into frames of length N. This is accomplished by multiplying the signal frame by the Hanning window function. Once the signal frame has been isolated, the FFT is computed on the windowed signal chunk, converting it from the time domain to the frequency domain. To process the next frame, the analysis window moves ahead by H (hop size) number of samples.

The output of the analysis step is a collection of M frequency bins (see Section 2.3), where each bin contains a complex value representing the magnitude and phase information for a particular frequency [11]. At this point, the signal frame can be modified in the frequency domain, such as altering the pitch.

4.2 Modification

In this section, I will explain how the phase vocoder is used to modify a given input audio signal. After the modification step, the audio signal will have been successfully pitch shifted in the frequency domain.

With the completion of the analysis step, the audio signal has been transformed into the frequency domain. The signal is now represented as linearly spaced frequency bins that range from 0 to $N/2+1$. Each bin k is a complex number of the form $a+jb$ that represents a specific analysis frequency [8]. An analysis frequency can be thought of as sinusoid that completes an integer number of cycles over an N sample window. Thus, each frequency bin k measures the contribution of a sinusoid at frequency $\frac{k}{N} * f_s$, where f_s is the sampling rate [8].

One shortcoming of the FFT however, is that it is possible for signals frequencies to fall between frequency bins. For example, a typical sampling rate might be 44100Hz, and let our window size be 1024 samples. To calculate the spacing between frequences we compute:

$$\Delta f = \frac{f_s}{N} = \frac{44100}{1024} \approx 43\text{Hz}$$

Therefore, at bin 0 the analysis frequency is 0Hz, at bin 1 its 43Hz at bin 2 its 86Hz and so on. If a signal's frequency falls between one of these bins, its energy is distributed accross all other bins and the actual frequency becomes obscured [6]. The energy distribution manifests itself as spectral leakage and as seen in Section 2.2 a windowing function helps to reduce the leakage.

4.2.1 Computing a More Accurate Frequency

To more accurately calculate a signal frame's frequency, the phase vocoder employs the use of the frame's phase information from the FFT. The core principle behind this computation is that frequency is the derivative of phase. Thus, by using a signal's phase from two different hops the exact frequency of the signal can be calculated. The equation below mathematically demonstrates the frequency of a signal ω written as a first order derivative in terms of phase ϕ:

$$\omega[n] = \frac{\phi[n] - \phi[n-H]}{H}. \tag{4}$$

Now, we can rearrage Equation 5 to isolate the phase difference:

$$\phi[n] - \phi[n-H] = \omega[n]H = \frac{2\pi kH}{N}. \tag{5}$$

Thus, the phase shift for a sinusoid with a frequency exactly matching a frequency bin over one hop size is given by Equation 7. What we now have is a basis to compare the actual signal frequency to. Computing the adjustment factor, also referred to as the phase remainder ϕ_r, we can subtract the ideal phase from the actual observed phase [6]:

$$\phi_r[n] = (\phi[n] - \phi[n-H]) - \frac{2\pi kH}{N}. \tag{6}$$

In order to keep from getting negative frequenceis, we must wrap ϕ_r between $(-\pi, \pi]$ [11]. Once the phase is wrapped, we can calculate the the deviation of the observed frequency from the bin frequency:

$$\omega[n] - \omega_k = \frac{\text{wrap}(\phi_r[n])}{H}, \text{ and} \tag{7}$$

$$\omega[n] = \frac{\text{wrap}(\phi_r[n])}{H} + \omega_k. \tag{8}$$

We now have an equation that computes the exact frequency of the sinusoid. Due to operating in a real-time enviornment however, it is more computationaly efficient to compute a fractionl FFT bin number, which represents the frequency in terms of where it would fall if the bins had infinite resolution. To compute the fractional FFT bin number, we only need to scale Equation 9 by a factor of $2\pi/N$ since $\omega[n] = \frac{2\pi k}{N}$. Doing so yields:

$$b[n] = \frac{\text{wrap}(\phi_r[n])N}{2\pi H} + k \tag{9}$$

and we now have a method to more accurately compute the frequency of our signal.

4.2.2 Pitch Shifting and Synthesizing a Signal

Now that we have a more accurate estimation of the frequency of the audio signal, pitch shifting the signal is as simple as scaling the frequency $b[n]$ by a pitch shift factor R. In

doing so, we compute a new fractional bin $b_s[n]$ where $b_s[n] = Rb[n]$. Finally we calculate the new bin number $k' = \text{floor}(Rk + 0.5)$ that will contain the shifted frequency.

The final part of the modification step is to calculate a new phase value for the shifted frequency. The new phase value ensures that the modified signal is reconstruced correctly and does not introduce any discontinuities at the signal frame's edges [1]. The calcuation is similar to Equation 6 with a few adjustments. To compute the new phase value, we are only concerned with how much the phase has changed from one hop to the next, thus we first must compute the phase remainder of our new frequency:

$$\phi_{rs}[n] = \frac{2\pi H(b_s[n] - k')}{N}. \tag{10}$$

Next, we add the ϕ_{rs} to the phase of the shifted signal that perfectly fits into a frequency bin to obtain:

$$\frac{2\pi H b_s[n] - 2\pi H k'}{N} + \frac{2\pi k' H}{N} = \frac{2\pi H b_s[n]}{N}. \tag{11}$$

Recall that $b_s[n]$ is the fractional bin index, hence we have now computed the phase of the actual frequency of the input signal frame. We can now add the previous hops phase to the result of Equation 11 to obtain:

$$\phi_s[n] = \text{wrap}\left(\phi_s[n - H] + \frac{2\pi H b_s[n]}{N}\right) \tag{12}$$

where $\phi_s[n]$ is the actual phase of this bin at the current hop.

4.3 Synthesis

Once the signal frame has been modified and a new phase value hase been calculated, the IFFT converts the signal back to the time domain. Then the output is reconstructed using the Constant Overlap-Add (COLA) paradigm. A detailed explanation of COLA can be found here [13]. The key idea is that if samples in the window function, offset by H, add to a constant C, then the window possesses the "Constant Overlap-Add Property" [13]. The mathematical expression below demonstrates the COLA property:

$$\sum_{m=-\infty}^{\infty} w(n - mH) = 1, \quad \forall n \in \mathbb{Z} \tag{13}$$

and Figure 4 below graphically shows the COLA Property.

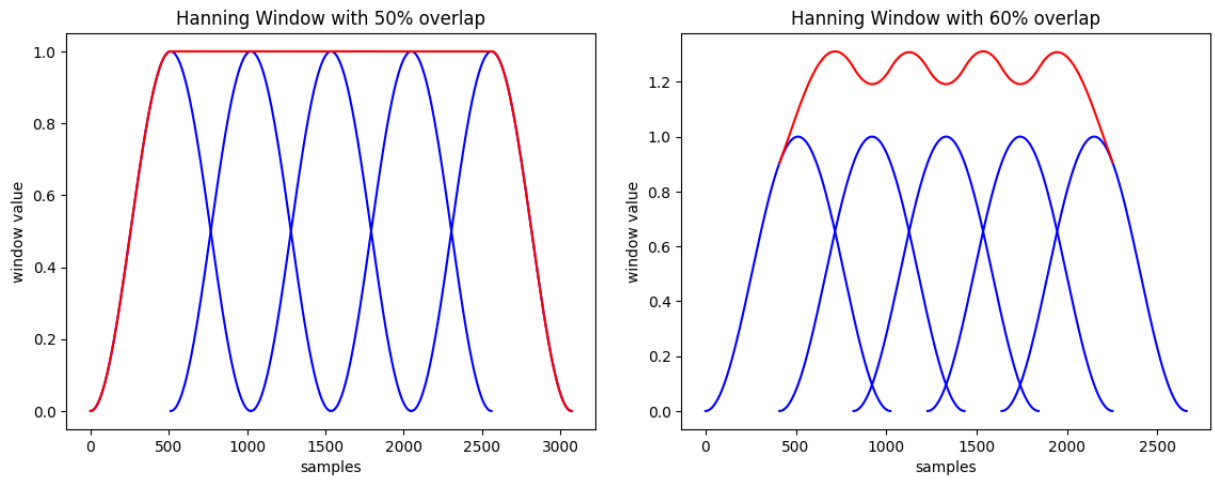

Figure 4: Illustration of the Constant Overlap-Add (COLA) property. The first graph demonstrates that a 50% overlap satisfies the COLA property. The second graph uses a 60% overlap which does not satisfy the COLA property.

This project employs the Hanning window function with a 75% overlap. Even though this amount of overlap does not perfectly possess the COLA Property, the benefit of using a Hanning window with a 75% overlap outweighs the negatives. As seen in section 4.3, the phase vocoder uses previous phase values to more accurately estimate a signal's frequency. Thus, by having a larger overlap, the phase vocoder more frequently analyzes the input signal which results in more accurate phase calculations [1].

To reconstruct the output signal, the values from the inverse STFT are first added into an output buffer. This buffer serves as temporary storage; it is not the final synthesized audio signal. The output buffer is necessary because, due to overlapping windows ensured by the COLA property, certain samples are calculated as the sum of multiple windows. Once the samples have been correctly added in the buffer, a write pointer is used to sequentially read them to the output audio buffer.

5 Methodology

The JUCE framework is used to implement the pitch-shifting audio plugin. JUCE is an open-source framework that supports the development of cross-platform audio applications and plugins. Its integrated build system allows projects to target multiple plugin formats, such as VST3, AU and more. JUCE projects are written in C++, making it well-suited for developing highly efficient, real-time audio plugins. Additionally, JUCE provides a wide range of built-in libraries that support common audio processing tasks, as well as tools for creating graphical user interfaces (GUIs).

The core components of JUCE we are most concerned with are the methods that handle an audio plugin's initialization, process and audio buffer's signal samples, and release the plugin's resources on termination. These built-in class methods provided by JUCE provide a basis for further development and require the developer to expand on them in order

to support their plugin's needs. The method that is used to handle an audio plugin's initialization is the PluginProcessor.PrepareToPlay() function. This callback method is invoked before the plugin starts to process audio [4]. The purpose of this method is to prepare the plugin for handling audio buffers and initialize any resources the plugin might need. Once an audio plugin has been initialized, the plugin will start to receive signal data, typically from a DAW. Processing an entire signal in a real-time environment is not possible, thus the plugin employs block based processing where it only receives a particular block (chunk) of a signal at a time. These blocks are referred to as audio buffers and they contain the input signal's samples. The audio samples are passed to the plugin's processBlock() method which is where the plugin performs any modification of the signal. Once the plugin has finished modifying the current audio buffer, JUCE passes the buffer back to the DAW where it can then be either thrasmited to speakers or be further processed by additional audio pugins. On termination, the plugin will call the PluginProcessor.ReleaseResources() method. This method ensures that the memory allocated during the plugin's life span is released and that the plugin did not cause a memory leakage.

6 Results

- A successful pitch-shift audio plugin will produce a shifted audio signal with minimal artifacts introduced
- Additionally, the success will depend on how well it can handle multiple notes. Full chords, two power chords, fast-alternate picking etc.
- If time permits, I will test the audio clarity using one of the tests found here: [9]

7 Conclusion

This project effectively demonstrates a working implementation of a phase vocoder based ptich shifter. It is able to both pitch up and down an octave (12 semitones), while maintaining reasonable audio clarity and introducing minimal artifacts. The solution follows the phase vocoder paradigm: converting the signal from the time domain to the frequency domain, using phase information obtained from this step to improve frequency estimations, modifying the signal in the frequency domain, and finally converting the signal back to the time domain.

While this implementation is good at shifting monophonic signals, it struggles at accurately pitch shifting polyphonic signals (multiple notes at once, chords). More advanced pitch shifters employ techniques such as peak-locking, zero-padding and advanced phase techniques [12]. One notable future improvement would be to examine alternative methods of phase reconstruction that result in a more complete signal after signal modification.

- need some closing sentence

References

[1] Barry, D., Dorran, D., and Coyle, E. Time and pitch scale modification: A real-time framework and tutorial.

[2] Dolson, M. The phase vocoder: A tutorial. *Computer Music Journal 10*, 4 (1986), 14–27.

[3] Götzen, A. D., Bernardini, N., and Arfib, D. Traditional (?) implementations of a phase-vocoder: The tricks of the trade. In *Proceedings of the COST G-6 Conference on Digital Audio Effects (DAFx-00)* (Verona, Italy, Dec. 2000), pp. 37–44.

[4] JUCE. Juce framework documentation. https://juce.com/learn/documentation/, 2024. Accessed: 2026-04-08.

[5] Laroche, J., and Dolson, M. New phase-vocoder techniques for pitch-shifting, harmonizing and other exotic effects. In *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* (New Paltz, NY, USA, Oct. 1999), IEEE, pp. 91–94.

[6] Lim, K. A. An open-source phase vocoder with some novel visualizations. Capstone project, School of Informatics, Indiana University Bloomington, June 2007.

[7] Lyons, R. G. *Understanding Digital Signal Processing*, 3rd ed. Pearson, Upper Saddle River, NJ, USA, 2010.

[8] McFee, B. Digital signals theory, 2025.

[9] Rai, A., and Barkana, B. D. Analysis of three pitch-shifting algorithms for different musical instruments. In *2019 IEEE Long Island Systems, Applications and Technology Conference (LISAT)* (2019), IEEE, pp. 1–6.

[10] RaNa, M. R. H. The influence of technology on modern music production. *International Journal of Humanities and Information Technology 6*, 04 (2024), 19–25.

[11] Reiss, J. D., and McPherson, A. *Audio Effects: Theory, Implementation and Application*. CRC Press, 2014. Accessed as PDF.

[12] Royer, T. Pitch-shifting algorithm design and applications in music. Master's thesis, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2019.

[13] Smith, J. O. *Spectral Audio Signal Processing*. 2011. Online book.